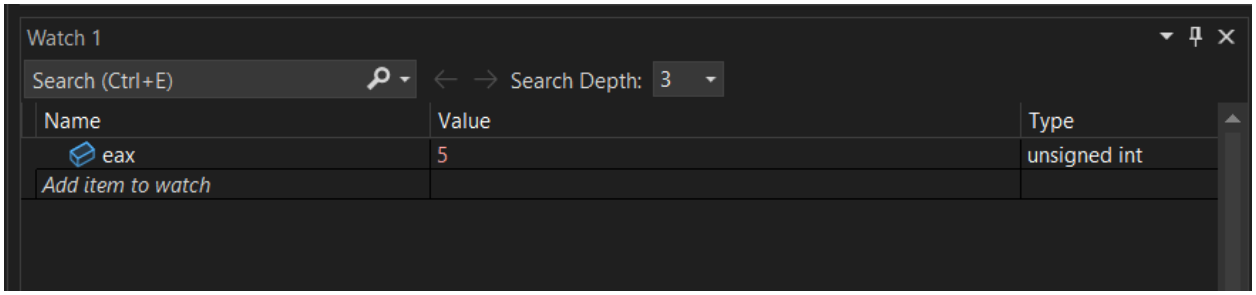
Task 1:

```
.code
main proc
        mov eax, 5
        inc eax
        dec eax

        INVOKE ExitProcess, 0
main endp

END main
```



Task 2:

```
.data
var1 sdword 5
var2 sdword 10
var3 sdword ?
.code
main proc

        mov eax, var1
        mov ebx, var2

        sub ebx,eax
        mov ecx,ebx

        mov eax, var1
        mov ebx, var2

        sub eax,ebx
        mov edx,eax

        add ecx,edx
        mov var3,ecx

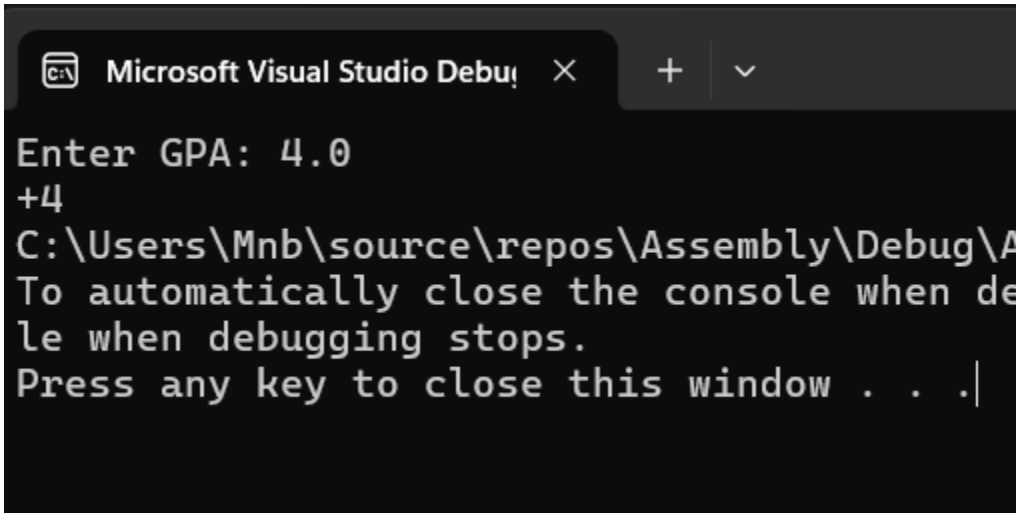
        INVOKE ExitProcess, 0
```

main endp

Name	Value	Type
eax	4294967291	unsigned int
ebx	10	unsigned int
ecx	0	unsigned int
edx	4294967291	unsigned int
var1	5	long
var2	10	long
var3	0	long

Task 3:

```
.code
main PROC
mov edx, offset msg1
call writestring
call readint
mov a, eax
call writeint
```



Task 4:

```
.data
a dd 10
b dd 20
d dd 30

.code
main PROC

mov eax, a
mov ebx, b
mov ecx, d
```

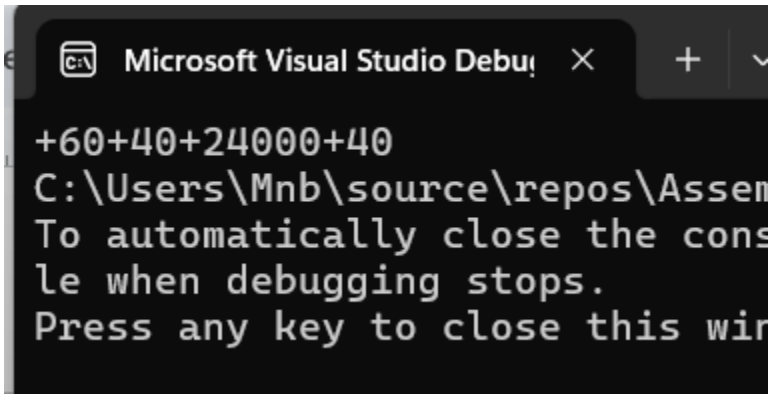
```
add ebx,ecx
add eax,ebx
call writeint

sub ebx,ecx
sub eax,ebx
call writeint

mul ebx
mul ecx
call writeint

div ebx
div ecx
call writeint

INVOKE ExitProcess,0
```



Task 5:

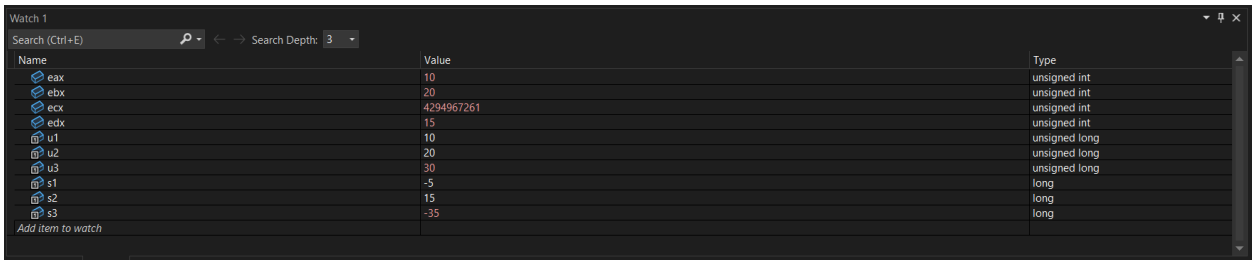
```
.data
u1 dword 10
u2 dword 20
u3 dword ?

s1 sdword -5
s2 sdword 15
s3 sdword ?
.code
main PROC

mov eax,u1
mov ebx, u2
add eax,ebx
```

```
mov u3, eax
sub eax,ebx

mov ecx, s1
mov edx, s2
sub ecx,edx
mov s3, ecx
sub ecx,edx
mov s3,ecx
```



Unsigned numbers are treated as normal but signed negative numbers are stored in two's complement form so they appear as very large numbers i.e value of ECX

Task 6:

```
.data
u1 dword 10
u2 dword 20
u3 dword ?

s1 sdword -5
s2 sdword 15
s3 sdword ?
.code
main PROC

mov eax,u1
mov ebx, u2
mul ebx
mov u3, eax
div ebx

mov eax, s1
mov edx, s2
mul edx
mov s3, eax
div ecx
mov s3,eax
```

When multiplying only the registers being used are changed (multiplicant and eax) however when dividing, eax is used for the quotient and edx is used for the remainder so both of them are changed even if edx is not directly used in the calculation. Edx is used by mul when the resultant of multiplication between two numbers is too large to be stored in eax

Task 7:

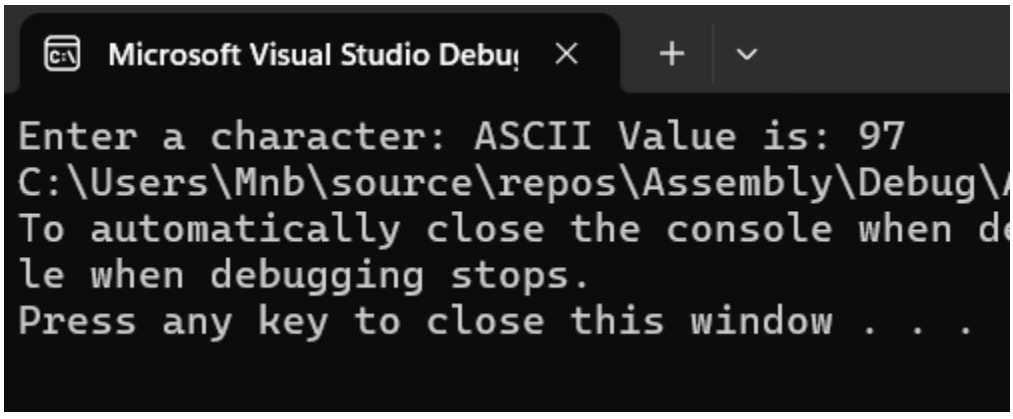
```
.data

msg1 BYTE "Enter a character: ",0
msg2 BYTE "ASCII Value is: ",0
char BYTE ?

.code
main PROC

mov edx,offset msg1
call writestring
call readchar
mov char,al
movzx eax,char


mov edx, offset msg2
call writestring
call writedec
```



ascii for a

Task 8:

```
.data
```

```
msg1 BYTE "Enter a number: ", 0
num1 DWORD ?
num2 DWORD ?

.code
main PROC

mov edx, offset msg1
call writestring
call readint
mov num1, eax

add eax,10
call writeint
sub eax,5
call writeint
mov ebx, 10
mul ebx
call writeint
mov ebx,2
mov edx, 0
div ebx

mov num2, eax
call writeint
```

